

STRATOS: Reference Manual

Taine J. Rossini

Version 1.0.0.

Abstract

This document lists every argument recognised by **stratos**, the post-shock relaxation solver, and the returned data.

Usage

Stratos has no command line interface. It is a single class, imported into a job script.

```
from stratos import Stratos

st = Stratos('air-5sp.gas', 'air-5sp.chem', '', 0.1, 1000, 6075.1,
             {"N2": 0.767, "O2": 0.233}, 365.5, rho1=0.02322)

columns = st.normal()
```

A run has three parts.

- Construct a **Stratos** object from the gas-models, domain, and pre-shock freestream.
- Call one solution method, **normal()**, **oblique()** or **shock_angle()**, to march along the streamline.
- Use the returned dictionary. Nothing is written to disk and nothing is plotted; every method hands back a dictionary of equal-length numpy arrays for the script to do with as it likes.

One object can be used for as many runs as needed. The freestream is fixed at construction, so a sweep over shock angles is a loop over the method, not over the constructor.

Constructor arguments

The arguments are positional, in this order, and none of the first eight has a default.

Argument	Type	Description
gasFile	string	Path to the gas-model file.
chemFile	string	Path to the reaction file.
kinFile	string	Path to the energy-exchange file, for two-temperature runs. Pass "" on a one-temperature gas-model.
x_end	float	Length of the marching domain, m.
cells	int	Number of points the domain is divided into.
u1	float	Freestream velocity, m/s.
massf1	dict or list	Freestream mass fractions. A dictionary is keyed by species name.
T1	float	Freestream temperature, K.

In addition, exactly one of `p1` or `rho1` must be supplied as a keyword.

Argument	Type	Default	Description
<code>p1</code>	float	None	Freestream pressure, Pa.
<code>rho1</code>	float	None	Freestream density, kg/m ³ .

If both are given, `p1` is used. If neither is, the construction fails.

Solution methods

`normal()`

Applies the normal-shock jump conditions to the freestream and marches from the immediate post-shock state to `x_end`. Returns the column dictionary.

```
columns = st.normal()
```

`oblique(theta)`

Solves the shock over a wedge of half-angle `theta`. The shock angle from the frozen relation is only a starting guess; a secant iteration adjusts it until the flow angle at the end of the domain matches the wedge. Returns a tuple of the column dictionary and the converged shock angle, in radians.

Argument	Type	Default	Description
<code>theta</code>	float	required	Wedge half-angle, radians.
<code>verbose</code>	bool	True	Prints the residual, last flow angle and shock angle each iteration.
<code>end_type</code>	string	'x_arc'	Which column <code>x_end</code> is measured along, and which the result is interpolated onto.
<code>max_iter</code>	int	30	Maximum secant iterations for the shock angle iteration.
<code>tol</code>	float	1e-5	Convergence tolerance on the flow angle, radians.

```
columns, beta = st.oblique(np.deg2rad(15.0), verbose=False)
```

`shock_angle(beta)`

Marches the streamline behind a shock of known angle `beta`, in radians. This is the method for tracing a shock whose shape is already known. Returns the column dictionary.

```
columns = st.shock_angle(beta)
```

The tangential velocity is preserved across the shock and the normal component marched, so this and `oblique()` share their post-processing and return the same columns, less `x_p`.

`interpolate_to_length(no_cells, target_length, columns, reference)`

Interpolates a result onto an evenly spaced grid. It can be used to re-grid a result afterwards, for example onto residence time rather than distance.

Argument	Type	Description
<code>no_cells</code>	int	Number of points in the new grid.
<code>target_length</code>	float	Upper limit of the new grid, in the units of <code>reference</code> .
<code>columns</code>	dict	The dictionary to re-grid. Every column in it is interpolated.
<code>reference</code>	string	The column that the new grid spans.

```
st.interpolate_to_length(1000, 2e-7, columns, 't')
```

Returned data

Every method returns a dictionary of numpy arrays, all `cells` long.

From `normal()`

Key	Units	Description
<code>x</code>	m	Distance behind the shock.
<code>t</code>	s	Time behind the shock, integrated along the streamline.
<code>u</code>	m/s	Velocity.
<code>T_tr</code>	K	Translational-rotational temperature.
<code>T_ve</code>	K	Vibro-electronic temperature. Two-temperature runs only.
<code>p</code>	Pa	Pressure.
<code>rho</code>	kg/m ³	Density.
<code>e_tr</code>	J/kg	Translational-rotational internal energy.
<code>e_ve</code>	J/kg	Vibro-electronic internal energy. Two-temperature runs only.
<code>gamma</code>	-	Ratio of specific heats.
<code>R</code>	J/kg/K	Gas constant of the mixture.
<code>Y_<species></code>	-	Mass fraction, one column per species.
<code>X_<species></code>	-	Mole fraction, one column per species.

On a one-temperature gas-model the two vibro-electronic columns are absent.

From `oblique()` and `shock_angle()`

Both carry every column above, except that `x` is removed in favour of the shock-aligned and Cartesian coordinates below, and `u` is redefined as the velocity magnitude.

Key	Units	Description
<code>x_n</code>	m	Distance normal to the shock.
<code>x_t</code>	m	Distance tangential to the shock.
<code>x_arc</code>	m	Distance along the streamline, the default <code>end_type</code> .
<code>x_coord</code>	m	Cartesian <i>x</i> , aligned with the freestream, with the origin where the streamline crosses the shock.
<code>y_coord</code>	m	Cartesian <i>y</i> , normal to the freestream.
<code>x_p</code>	m	Distance along the wedge surface. <code>oblique()</code> only.

Key	Units	Description
u_n	m/s	Velocity normal to the shock, which is what is marched.
u_t	m/s	Velocity tangential to the shock, constant across it.
u	m/s	Velocity magnitude, not the normal component.
flow_angle	rad	Angle of the streamline, measured from the freestream.

A complete job script

A one-temperature normal shock, marched a metre downstream.

```
# Load libraries.
from stratos import Stratos
import numpy as np

# Construct the solver.
st = Stratos('air-7sp.gas', 'air-7sp.chem', '', 1.0, 100000, 4273.64,
             {"N2": 0.78, "O2": 0.22}, 300, 133.3)

columns = st.normal()

# Save the result to a CSV file.
np.savetxt('profile.csv', np.column_stack(list(columns.values()))),
           delimiter=',', header=','.join(columns.keys()), comments='')
```

A sweep of wedge angles on the same object.

```
# Load libraries.
from stratos import Stratos
import numpy as np

# Construct the solver.
st = Stratos('air-5sp.gas', 'air-5sp.chem', 'air-5sp.kin', 0.1, 1000, 6075.1,
             {"N2": 0.767, "O2": 0.233}, 365.50, rho1=0.02322)

results = {}

for angle in [5, 10, 15, 20, 25, 30, 35, 40]:
    results[angle], _ = st.oblique(np.deg2rad(angle), verbose=False)

# Print the results at the end of the domain.
for angle, columns in results.items():
    print(angle, columns['T_tr'][-1])
```